

计算机组成原理 Project1 实验报告（Week 1）

8 位定点卷积-激活-池化链运算器设计

鲁昕宁 2414015

2026 年 4 月 26 日

<https://github.com/SidneyLu/Computer-Organization-and-Design-NKU-CS/tree/master/project1>

1 实验目标

本实验面向边缘端 AI 场景，设计并验证一个 8 位定点卷积-激活-池化链运算器。工程目标如下：

- 输入为 5×5 的 8 位无符号灰度矩阵，像素范围为 $0 \sim 255$ ；
- 运算流程为“ 5×5 输入 $\rightarrow 3 \times 3$ 卷积层 $\rightarrow$ ReLU 激活层 $\rightarrow 2 \times 2$ 平均池化层”；
- 输出为 2×2 的 8 位特征值矩阵；
- 在 Vivado 环境下完成模块设计、功能仿真、综合与时序分析，并比较不同优化方案的资源占用与性能。

2 实验环境

本工程的开发与验证环境如下：

- FPGA 器件：Xilinx Artix-7 xc7a200tfbg676-2
- RTL 语言：Verilog HDL
- 功能仿真：Vivado 2025.1 / XSIM
- 综合与时序分析：Vivado 批处理脚本 `scripts/run_vivado_multi_synth.tcl`
- 时钟约束：`create_clock -period 5.000 -name clk [get_ports clk]`，即 200 MHz
- I/O 约束：`set_input_delay 0.000` 与 `set_output_delay 0.000`
波形保留策略也在工程中固化：
- XSIM 原始波形数据库保存在 `build/xsim/waves/*.wdb`；
- 便于二次处理的文本波形保存在 `build/xsim/waves/*.vcd`；
- 报告中的代表性波形图由 `scripts/export_wave_figures.py` 从 VCD 自动导出到 `report/figures/`。

3 实验原理

3.1 卷积运算

输入图像记为 X ，卷积核记为 K 。本实验采用 3×3 卷积核对 5×5 输入做 valid 卷积，得到 3×3 特征图：

$$Y(i, j) = \sum_{u=0}^2 \sum_{v=0}^2 X(i+u, j+v) \cdot K(u, v), \quad 0 \leq i, j \leq 2$$

其中像素 $X(i, j)$ 为 8 位无符号数，卷积核权重 $K(u, v)$ 为 8 位有符号数。

3.2 ReLU 激活

卷积输出经过 ReLU 激活后，负数被截断为 0，正数保留：

$$R(i, j) = \max(0, Y(i, j))$$

为保证最终结果能安全落回 8 位无符号范围，本工程进一步加入上饱和处理：
若 $R(i, j) > 255$ ，则输出钳位为 255。

3.3 平均池化

对 3×3 激活图执行 2×2 平均池化，输出 2×2 结果：

$$P(i, j) = \left\lfloor \frac{R(i, j) + R(i, j + 1) + R(i + 1, j) + R(i + 1, j + 1)}{4} \right\rfloor, \quad 0 \leq i, j \leq 1$$

由于分母恒为 4，硬件上可直接用右移 2 位实现，因此无需额外的通用除法器。

3.4 实验分组

本次实验实现了 5 个独立顶层：

1. 串行：cnn_compare_base_top
2. 串行 + BoothWallace 乘法器优化：cnn_compare_booth_top
3. BoothWallace 乘法器优化 + SIMD：cnn_compare_booth_simd_top
4. BoothWallace 乘法器优化 + SIMD + 流水线：cnn_compare_booth_simd_pipe_top
5. 第 4 组基础上换用 DSP：cnn_compare_booth_simd_pipe_dsp_top

4 里程碑 1：核心模块设计与功能验证

4.1 基础模块设计

里程碑 1 首先完成了 4 个基础模块的串行版本：

- mul8: 8 位无符号像素与 8 位有符号卷积核的乘法；
- add8: 8 位无符号加法；
- relu_s20: 20 位有符号卷积结果的 ReLU 与 8 位饱和截断；
- div4: 10 位无符号数除以 4 的右移实现。

下面在模块首次介绍的位置给出对应的完整 Verilog 源码。

模块 mul8 的 Verilog 源码

对应文件：../src/baseline/mul8.v

```
module mul8 (  
    input  wire [7:0] a_u8,  
    input  wire signed [7:0] b_s8,  
    output wire signed [15:0] p_s16
```

```
);
    assign p_s16 = $signed({1'b0, a_u8}) * b_s8;
endmodule
```

模块 add8 的 Verilog 源码

对应文件: ../src/baseline/add8.v

```
module add8 (
    input  wire [7:0] a_u8,
    input  wire [7:0] b_u8,
    output wire [7:0] y_u8
);
    assign y_u8 = a_u8 + b_u8;
endmodule
```

模块 relu_s20 的 Verilog 源码

对应文件: ../src/baseline/relu_s20.v

```
module relu_s20 (
    input  wire signed [19:0] x_s20,
    output wire [7:0] y_u8
);
    wire signed [19:0] x_clip_low;
    assign x_clip_low = (x_s20 < 0) ? 20'sd0 : x_s20;
    assign y_u8 = (x_clip_low > 20'sd255) ? 8'd255 : x_clip_low[7:0];
endmodule
```

模块 div4 的 Verilog 源码

对应文件: ../src/baseline/div4.v

```
module div4 (
    input  wire [9:0] x_u10,
    output wire [7:0] y_u8
);
    assign y_u8 = x_u10[9:2];
endmodule
```

在此基础上，又按计划补齐了优化所需模块：

- mul8_booth_wallace: 真实的基 4 Booth 编码 + Wallace 树压缩 8 位乘法器；
 - simd_mul8x4: 4 路 BoothWallace lane 封装；
 - simd_add8x4、simd_relu8x4、simd_avgpool2x2_8bit: 分别承担 4 路加法、4 路激活与池化求均值；
 - simd_mul8x4_dsp 与 mul8_dsp48e1: 作为第 5 组 DSP 版的乘法实现。
- 优化模块源码与仿真结果详见第三部分。

4.2 模块级仿真

本实验所有仿真均通过 Vivado XSIM 完成。统一命令为：

```
powershell -ExecutionPolicy Bypass -File .\scripts\run_vivado_sim.ps1 -Mode  
↪ all
```

其中基础模块验证可单独使用：

```
powershell -ExecutionPolicy Bypass -File .\scripts\run_vivado_sim.ps1 -Mode  
↪ basic
```

模块级 testbench 的覆盖范围如表 1 所示。

表 1: 里程碑 1 模块级 testbench

testbench	验证对象	覆盖方式
tb_mul8	mul8	穷举 65536 组 “像素 $\times$ 权重” 输入
tb_add8	add8	穷举 65536 组 8 位无符号加法输入
tb_relu_s20	relu_s20	负数、线性区间、饱和区间与边界值
tb_div4	div4	穷举全部 1024 组 10 位输入

Vivado 日志显示上述 testbench 均通过：

- tb_mul8 PASS
- tb_add8 PASS
- tb_relu_s20 PASS
- tb_div4 PASS

按照验证对象顺序，下面给出里程碑 1 的基础模块 testbench 源码。

testbench tb_mul8 的 Verilog 源码

对应文件：../sim/tb_mul8.v

```
`timescale 1ns/1ps  
  
module tb_mul8;  
    reg [7:0] a_u8;  
    reg signed [7:0] b_s8;  
    wire signed [15:0] p_s16;  
  
    integer ia;  
    integer ib;  
    reg signed [15:0] expected;  
  
    mul8 dut (
```

```

        .a_u8(a_u8),
        .b_s8(b_s8),
        .p_s16(p_s16)
    );

    initial begin
        for (ia = 0; ia < 256; ia = ia + 1) begin
            for (ib = -128; ib < 128; ib = ib + 1) begin
                a_u8 = ia[7:0];
                b_s8 = ib[7:0];
                expected = $signed({1'b0, a_u8}) * b_s8;
                #1;
                if (p_s16 != expected) begin
                    $display("tb_mul8 FAIL a=%0d b=%0d got=%0d exp=%0d",
                        ia, ib, p_s16, expected);
                    $finish;
                end
            end
        end

        $display("tb_mul8 PASS");
        $finish;
    end
endmodule

```

testbench tb_add8 的 Verilog 源码

对应文件: ../sim/tb_add8.v

```

`timescale 1ns/1ps

module tb_add8;
    reg [7:0] a_u8;
    reg [7:0] b_u8;
    wire [7:0] y_u8;

    integer ia;
    integer ib;
    reg [8:0] expected;

    add8 dut (
        .a_u8(a_u8),
        .b_u8(b_u8),
        .y_u8(y_u8)
    );

    initial begin

```

```

        for (ia = 0; ia < 256; ia = ia + 1) begin
            for (ib = 0; ib < 256; ib = ib + 1) begin
                a_u8 = ia[7:0];
                b_u8 = ib[7:0];
                expected = a_u8 + b_u8;
                #1;
                if (y_u8 != expected[7:0]) begin
                    $display("tb_add8 FAIL a=%0d b=%0d got=%0d exp=%0d",
                        ia, ib, y_u8, expected[7:0]);
                    $finish;
                end
            end
        end
        end
        $display("tb_add8 PASS");
        $finish;
    end
endmodule

```

testbench tb_relu_s20 的 Verilog 源码

对应文件: ../sim/tb_relu_s20.v

```

`timescale 1ns/1ps

module tb_relu_s20;
    reg signed [19:0] x_s20;
    wire [7:0] y_u8;

    integer iv;
    integer expected;

    relu_s20 dut (
        .x_s20(x_s20),
        .y_u8(y_u8)
    );

    task automatic check_value;
        input integer val;
        input integer exp;
        begin
            x_s20 = val;
            #1;
            if (y_u8 != exp[7:0]) begin
                $display("tb_relu_s20 FAIL x=%0d got=%0d exp=%0d", val,
                    ↵ y_u8, exp);
                $finish;
            end
        end
    endtask
endmodule

```

```

        end
    end
endtask

initial begin
    check_value(-524288, 0);
    check_value(-300, 0);
    check_value(-1, 0);
    check_value(0, 0);
    check_value(1, 1);
    check_value(127, 127);
    check_value(255, 255);
    check_value(256, 255);
    check_value(1024, 255);
    check_value(524287, 255);

    for (iv = -300; iv <= 600; iv = iv + 1) begin
        if (iv < 0) begin
            expected = 0;
        end else if (iv > 255) begin
            expected = 255;
        end else begin
            expected = iv;
        end
        check_value(iv, expected);
    end

    $display("tb_relu_s20 PASS");
    $finish;
end
endmodule

```

testbench tb_div4 的 Verilog 源码

对应文件: ../sim/tb_div4.v

```

`timescale 1ns/1ps

module tb_div4;
    reg [9:0] x_u10;
    wire [7:0] y_u8;

    integer ix;
    reg [7:0] expected;

    div4 dut (
        .x_u10(x_u10),

```

```

        .y_u8(y_u8)
    );

    initial begin
        for (ix = 0; ix < 1024; ix = ix + 1) begin
            x_u10 = ix[9:0];
            expected = ix[9:2];
            #1;
            if (y_u8 != expected) begin
                $display("tb_div4 FAIL x=%0d got=%0d exp=%0d", ix, y_u8,
                    ↪ expected);
                $finish;
            end
        end
        end

        $display("tb_div4 PASS");
        $finish;
    end
endmodule

```

优化模块的 testbench (tb_mul_compare、tb_simd_mul_compare) 源码与仿真结果详见第三部分。

4.3 代表性基础模块波形

下面给出 4 个基础模块的代表性波形，按模块验证顺序依次展示。

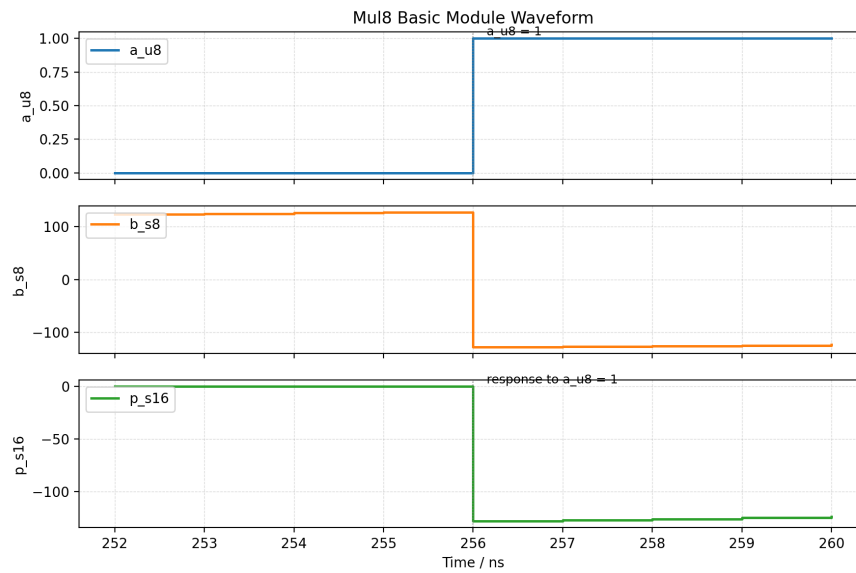


图 1: 基础模块波形: mul8 的乘法响应

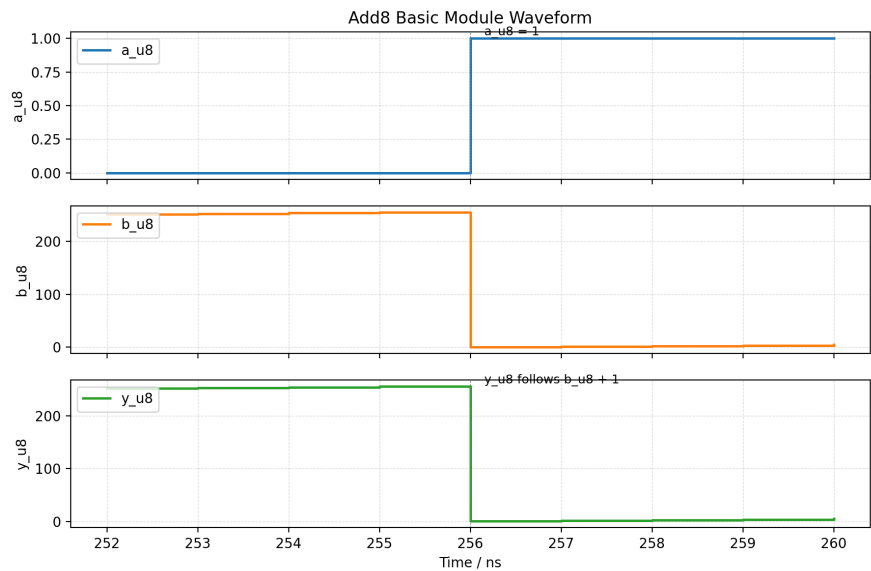


图 2: 基础模块波形: **add8** 的加法响应

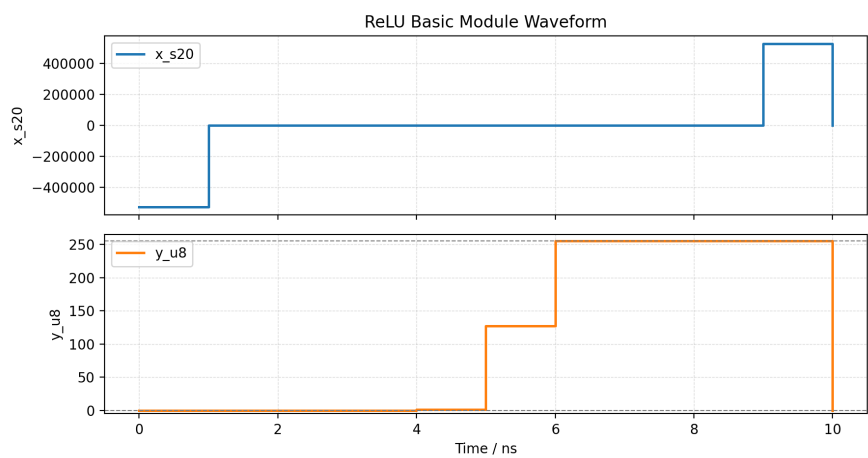


图 3: 基础模块波形: **relu_s20** 的截断与饱和行为

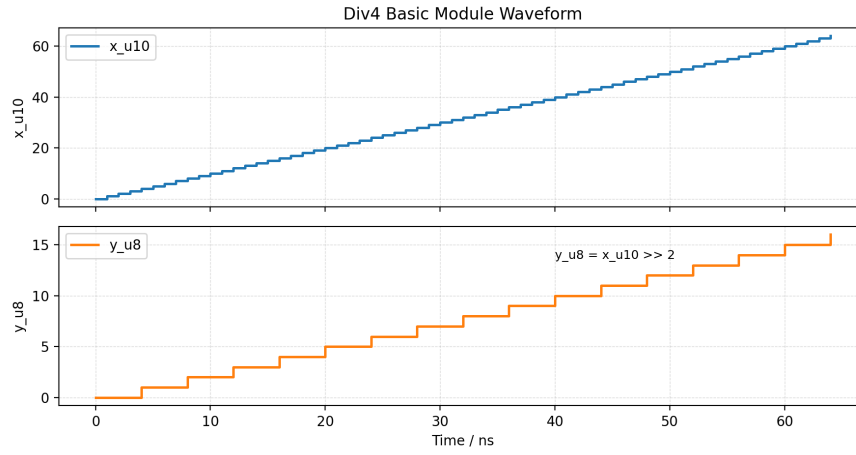


图 4: 基础模块波形: `div4` 的右移除 4 行为